Generalizing CNNs: Images → General Graphs

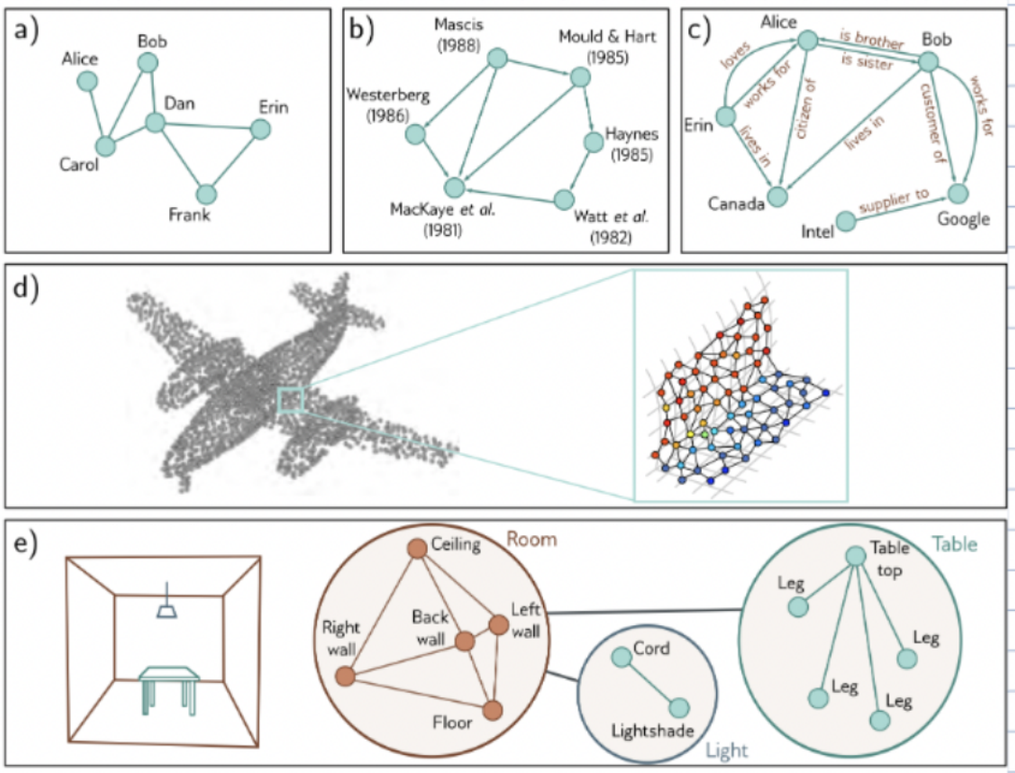


Fig 13.2 in Prince

The question I asked here was 'how are gnns and cnns similar'

1

CNNs are a *special case* of GNNs.

- A 2D image can be seen as a grid graph: each pixel = node, edges connect neighboring pixels (4- or 8-neighborhood).
- A CNN layer applies the same local filter at every location.
  This is equivalent to a GNN layer where each node updates its feature by aggregating features from its neighbors with *shared weights*.

Formally, a typical GNN message-passing layer is

$$h_i' = \sigma\Big(W_{\text{self}} h_i + \sum_{j \in \mathcal{N}(i)} W_{\text{msg}} h_j\Big).$$

It even derived the equation we end up using

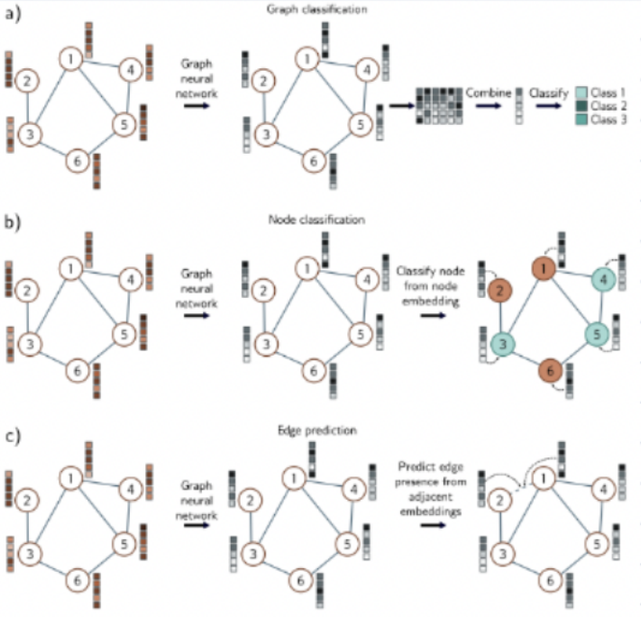
If $\mathcal{N}(i)$ are the pixels in a fixed $k \times k$ window around i and the weights W_{msg} are arranged as a convolution kernel, this becomes a standard CNN convolution.

So:
- CNN = GNN on a regular lattice graph (image grid) with fixed neighborhood and translation-invariant weights.
- GNN = generalization of CNN to *arbitrary* graphs (social networks, meshes,
scene graphs, etc.), where the same idea—local neighbor aggregation with shared parameters—is applied without assuming a grid structure.

Image Classification

Semantic Segmentation

Fig 13.6 in Prin



This is exactly what was mentioned in lecture: cnns are gens on a fixed structure

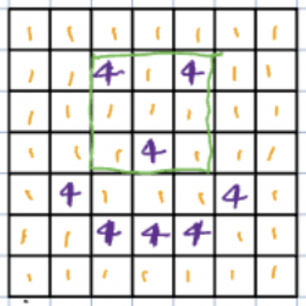
First Conceptual Step: Image as

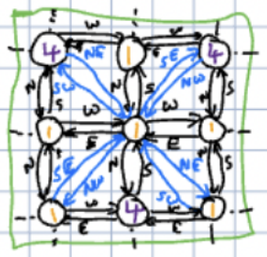
Nodes = Pixels

Directed Graph

First Conceptual Step: Image as a graph

Nodes = Pixels





Directed Graph
Edges labeled with one of
{N, S, E, W, NE, SE, NW, SW}

(Always a special relation to self)

A 3×3 Conv can be viewed as receiving messages from neighbors, each of whom has their own labeled edge to me, multiplying by an edge-label-specific weight matrix, and adding those up together with a special self-specific matrix acting on this node's info and a bias $\vec{b}$.

Recall Picture:

3×3 example

W_{NW}	W_N	W_{NE}
W_W	W_{MC}	W_E
W_{SW}	W_S	W_{SE}

c_{in}

c_{out} { $\vec{W}_i$, $\vec{b}$ } c_{out}

$$\vec{h}_{out} = \vec{b} + \sum_i W_i \vec{h}_{in,i}$$

Note: Implicit Zero-Padding and Same
Weight-sharing across nodes
1×1 Convs are local to a node

Matrices W_i and biases $\vec{b}$ are specific to layer of pixels

Taking Stock: We have, in principle, an immediate recipe to generalize CNNs to directed graphs with the same property: Every edge labeled from a discrete finite set.

What Generalizes Immediately: Weight-sharing (Nodes do the same things)
3×3 convs (As above, Matrices for edge-labels)
Additional Channels, Residual Connections, Dropout for MLPs, Stochastic

What does not generalize: Pooling / Downsampling and Upsampling

What needs discussion: Normalization.

Core Insight: The graph plays the role of the empty grid of pixels in CNN
The input is a graph with vectors in it

Here i asked to explain the picture on the left

Think of this as zooming in on **one output neuron** of a convolution (for a single spatial location).

1. 3×3 example
The 3×3 grid of w's is a single filter's spatial footprint:

$$\begin{matrix} w_{\mathrm{NW}} & w_{\mathrm{N}} & w_{\mathrm{NE}} \\ w_{\mathrm{W}} & w_{\mathrm{MC}} & w_{\mathrm{E}} \\ w_{\mathrm{SW}} & w_{\mathrm{S}} & w_{\mathrm{SE}} \end{matrix}$$

The ability to generate structured text in this format to visualize the 2d layer is very cool

- These are the weights that multiply the 3×3 patch of the input around some pixel (center = MC).
- In a real conv layer, we also have **input channels** c_{in} (e.g. RGB or feature maps).
For each channel, there is a full 3×3 grid of weights.
So together they form a set $\{W_i\}$ of weight matrices, one for each input channel i; their number is c_{in}.

2. Input vectors per position
At the same spatial location (e.g., MC) in the input feature map, you have a vector of activations across channels:

$\vec{h}_{\mathrm{in},i}$ (the feature vector on channel i u

This response is very detailed and much longer than what I would have wanted for annotations for my notes

What does not generalize: Pooling / Downsampling and Upsampling

Key Challenge

Why do we pool anyway? — Grow Receptive Fields Faster (with layers)

Hack for GNNs: Add a fake global node connected to everything.

Aside: "Oversquashing" — information from far away lost because of small embeddings.
vs "oversmoothing" — the operations of the GNN naturally blur away all fine-grained details

Is there a way to go: Many → Few based on vector contents

Like Clustering!

Recall Classic Clustering Approach: k-means / Lloyd's Algorithm for V

Hyperparam Choice: k

0) Start at random in some way

1) Calculate Centers

2) Map points to ce

Very detailed and accurate explanation, similar to what was said in class

For each spatial offset (NW, N, ..., SE), you get such a vector; the picture just focuses on one generic offset and calls its activation vector $\vec{h}_{\text{in},i}$.

3. Output computation
The output activation vector at that location, $\vec{h}_{\text{out}}$, is computed like a linear layer applied to all these inputs:

$$\vec{h}_{\text{out}} = \vec{b} + \sum_i W_i \vec{h}_{\text{in},i}.$$

- $\vec{b}$: bias vector for this output channel (or position).
- W_i: weight matrix associated with the i-th input (here: the weights at a particular offset and over channels).
- The sum over i: add up contributions from all relevant input channels / positions.

So the key idea: **a convolution at one output location is just a linear map of all input features within the receptive field** (the 3×3 patch × all channels), plus a bias.

Recall Classic Clustering Approach: k-means / Lloyd's Algorithm for VQ

Hyperparam Choice: k

0) Start at random in some way

1) Calculate Center

2) Map points to centers to get clusters

3) Goto 2 & Repeat

Is this differentiable? No.

What's the barrier? Hard Assignments of points to clusters

Can we use a softmax or something like that?

The question I asked here was 'What is Lloyd's Algorithm'

3

Lloyd's algorithm = the standard iterative procedure for k-means (and vector quantization):

1. Initialize k centers $\{\mu_j\}$ (e.g., randomly).
2. **Assignment step:** for each data point x_i, assign it to the closest center $c_i = \arg\min_j \|x_i - \mu_j\|^2$.
3. **Update step:** recompute each center as the mean of its assigned points $\mu_j = \frac{1}{|C_j|} \sum_{i:c_i=j} x_i$.
4. Repeat 2–3 until convergence.

Here, it's being recalled as the "classic clustering / VQ" algorithm. The key point in the slide is that the **hard assignment** in step 2 (the $\arg\min$) is non-differentiable, which is a barrier to using it inside gradient-based neural nets. That's why the note below asks whether we can replace it with something smooth like a softmax to get a differentiable, "soft" version of Lloyd's / k-means.

As we will see now and in future responses, the LLM is able to comprehend 'limitations of certain approaches' in various deep learning methods.

Here, it is clear that K-means does not have a differentiable step 2

Quick Idea: When we downsample, we get a smaller graph

i.e. $s_i^T = [p_1, \ldots, p_k]$ for node i

Need differentiable operation that does this.

Ingredients? 1) Softmax on scores will give me

I asked 'explain diffpool'

New Problem: Given a graph, do some differentiable steps to get

Don't know what to do. Make it learnable.

What's a generic way to have a learnable algorithm on a graph

USE a GNN!

Wherever we need to "downsample" a graph insert a little GNN that doesn't share weights with the rest of the GNN and has the structure:

But these $\vec{S}_i^T$ aren't a graph... Not yet.

This lecture was confusing to me back then, especially diff pool, but reading these thorough responses clears things up as each step is stated in extreme detail.

What do we do with these soft assignments to clusters?

Need New Vectors (Embeddings) for the k clusters.

$$\vec{h}_j^{(l+1)} = \sum_i S_i[j]\, \vec{h}_i^{(l)}$$

j^{th} cluster in layer $l+1$ (After Pooling)

Pre-Pooling nodes

"Probability"

The content of original node flows into different clusters according to membership

DiffPool is a *learnable, differentiable graph pooling* method.

What problem it solves
- Standard GNNs need a way to **downsample / coarsen graphs** (like pooling in CNNs) to build hierarchical representations for tasks such as graph classification.
- Classical graph coarsening isn't differentiable and often uses hard, fixed clustering rules, so it can't be trained end-to-end with a GNN.

How it solves it
- For each pooling layer, DiffPool learns an **assignment matrix**

$$S \in \mathbb{R}^{n \times k}$$

where S_{ij} is the (soft) probability that node i belongs to cluster j.
- S is produced by a small GNN followed by a row-wise softmax, so it's fully differentiable.
- Given node embeddings X and adjacency A, the pooled (coarsened) graph is

$$X' = S^\top X \quad \text{(cluster embeddings)}$$

$$A' = S^\top A S \quad \text{(cluster–cluster edges)}$$

- This can be stacked in multiple layers to form a hierarchy, and the whole model is trained jointly with the task loss (plus auxiliary losses in the paper).

So: DiffPool provides a **generic, end-to-end trainable way to cluster nodes and produce smaller graphs inside a GNN**, i.e., differentiable graph downsampling.

Property 1: Have clustering respect graph topology

$$\|A - {}_{N}S^{k}\,{}_{k}S^{N\,T}\|_F$$

What kind of information does this encourage in the embeddings?

I used a screenshot of property 1 and asked to explain

$A =$

$${}_{N}S^{k}\,{}_{k}S^{N\,T}$$ has low-rank structure

That's what we want for the example here

Note: The entries in SS^T are pairwise inner prod of the $\vec{s}_i^{\,T}$ with $\vec{s}_j^{\,T}$.

Property 2: Want to be close to hard cluster assignments

$$\frac{1}{N}\sum_i H(\vec{s}_i^{\,T})$$

Entropy minimized by $[0, 0, .., 1, 0, .. 0]$

Probabilities

In class it was mentioned that the derivation/development of using SS^T as our approximation has some intuition, but these notes annd annotations don't provide any intuition for where SS^T came from. Maybe if I was more specific in my prompt, the LLM would provide a response related to this question

Note: Hyperparameters: weights for Aux losses. Follow what others do...

	Edge Labels (finite)	Edge Labels (Infinite)	None
Directed	✓ Like CNN		
Undirected		Diff Pool results in weights on edges (in particular scalars)	✓ without Pooling

The expression in red is the loss term

$$\|A - SS^\top\|_F,$$

where:

- A is the graph adjacency matrix ($A_{ij} = 1$ if nodes i and j are connected, 0 otherwise).
- $S \in \mathbb{R}^{n \times k}$ contains the node embeddings (each row $\vec{s}_i$ is the embedding of node i).
- $SS^\top$ is an $n \times n$ matrix of pairwise inner products:

$$(SS^\top)_{ij} = \vec{s}_i^{\,\top}\vec{s}_j.$$

Key idea: we want $SS^\top$ to approximate A.

- If $A_{ij} = 1$ (edge present), minimizing the loss pushes $\vec{s}_i^{\,\top}\vec{s}_j$ to be large, so $\vec{s}_i$ and $\vec{s}_j$ become similar in the embedding space.
- If $A_{ij} = 0$ (no edge), the loss pushes $\vec{s}_i^{\,\top}\vec{s}_j$ to be small, so their embeddings become dissimilar or nearly orthogonal.

Thus, the embeddings are encouraged to **respect graph topology**: neighbors in the graph cluster together in the embedding space.

Low-rank structure:
A is $n \times n$, but S is $n \times k$ with $k \ll n$, so $SS^\top$ has rank at most k. Forcing $A \approx SS^\top$ therefore imposes a **low-rank approximation** of the adjacency matrix. This captures the idea that the graph can be explained by a small number of latent factors or communities (clusters). Each node's embedding $\vec{s}_i$ encodes its membership in these latent clusters, and pairwise similarities of embeddings ($\vec{s}_i^{\,\top}\vec{s}_j$) reconstruct the observed edges.

Final Challenge: Held-Out Data (Node-level Problems)

Final Challenge: Held-Out Data (Node-level Problems)

Approach 1: Excise entire nodes from the graph.

Challenge: Impacts connectivity

I took a screenshot of the entire slide and asked to explaion

Approach 2: Remove content from the node but leave it in.

? = learnable vector usually called MASK

Note: (From Office Hours)

By making ? learnable, we need to make sure the training examples seen during training let that happe

There are two desiderata from ?:

1) It shouldn't impede information flow as messages are passed. ← This will happen even with the in truly held-out positions during training.

2) It should prompt the network to correctly predict labels for the node even though there isn't input then

To get (2) to happen, it is useful to do data augme during training. Randomly mask some nodes for whi we have target labels, and keep the loss on that node' predictions.

Approaches and ideas usually aren't mentioned in any texts, readings, or papers. Usually, only the final result/best approach is demonstrated. These annotations being able to elaborate on these approaches provides a broader perspective on how to approach deep learning ideas

5

This slide is about *how to define a train/test split for node-level prediction on graphs* (e.g., node classification).

Goal: Evaluate on "held-out" nodes while still using the rest of the graph.

Approach 1: Excise entire nodes from the graph

- Idea: For test nodes, remove them completely—delete the node and all its edges.
- Problem: This **changes the graph's connectivity**:
  - neighbors lose edges,
  - paths disappear,
  - message passing (e.g., in a GNN) now operates on a different graph than the real one.
- So you are no longer testing in the same structural environment; the evaluation may be unrealistic.

Approach 2: Keep nodes, remove their content

- Instead of deleting test nodes, **keep them in the graph** with all their edges, but **remove / mask their features**.
- Concretely, for a node v with feature vector x_v, during training we replace it by something like

$$x'_v = \mathbf{0} \quad \text{or} \quad x'_v = m \odot x_v,$$

where m is a *learnable* or fixed **mask vector** (this is what the note "learnable vector usually called MASK" refers to).
- The node is still present so:
  - it can send/receive messages structurally,
  - but the model cannot "cheat" by directly reading its true features in training, since those are hidden or replaced.

This way, you preserve **graph connectivity** for message passing while still having **held-out node content**, which better matches the intended generalization setting.

RNNs: Recurrent Neural Nets

History: Before 2018, concensus view: Two big successes in NN:
Images & Vision using CNNs
Language & Speech using RNNs

Key issue in language & speech (along with control, time-series, etc.) sequenti

A signal-processing perspective